Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, contrôle de flot, syntaxe C et prévention des boucles infinies

Synthèse exécutive : Ce cours aborde de façon approfondie la mise en œuvre des structures répétitives (boucles) en algorithmique et en langage C. L'objectif fondamental est d'automatiser le traitement répété d'instructions sans duplication de code, en stricte conformité avec le principe DRY (Don't Repeat Yourself). Le cours établit la distinction entre l'évaluation à pré-condition (boucle while), l'évaluation à post-condition (boucle do-while) et l'itération déterminée à en-tête condensé (boucle for). Il met en lumière les conditions d'arrêt, la gestion systématique de la variable de contrôle pour prévenir les boucles infinies, la portée des variables et les règles d'écriture exigées aux évaluations de l'IUT (norme C, cartouche, tripartition et style Allman).

1. Utilité des structures itératives et principe DRY

En programmation algorithmique et en langage C, la répétition d'instructions ne doit jamais résulter d'un copier-coller de lignes de code. Si l'on doit traiter 10 000 éléments d'un tableau ou réitérer une demande de saisie jusqu'à obtenir une valeur valide, dupliquer le code détruit la maintenabilité du programme, augmente inutilement la taille du binaire exécutable et constitue une source massive d'erreurs. Les structures itératives (ou boucles) permettent d'exécuter de façon répétée un même bloc d'instructions tant qu'une condition logique (expression booléenne) reste vérifiée. Le point névralgique de la conception d'une boucle réside dans la gestion de sa fin. Une boucle robuste exige trois éléments indissociables : une initialisation correcte des variables de contrôle avant la boucle, une condition de maintien/sortie claire, et une mise à jour explicite de la variable de contrôle dans le corps de la boucle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci conserve sa valeur initiale Vraie : le programme entre alors dans une 'boucle infinie', saturant le processeur et gelant l'application.

- Principe DRY (Don't Repeat Yourself) : Règle d'ingénierie interdisant la duplication manuelle de lignes de code pour répéter une tâche.
- Condition d'arrêt : Expression booléenne évaluée avant ou après chaque itération déterminant la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : Variable modifiée au sein du corps de la boucle pour faire évoluer la condition logique vers Faux.
- Boucle infinie : État critique de blocage de l'exécutable provoqué par une condition d'arrêt qui demeure indéfiniment Vraie.

Cas concret enseignant : *Le professeur compare la boucle au fonctionnement d'un tapis roulant d'une usine d'emballage : tant qu'un colis est détecté par le capteur optique (condition vraie), le bras robotisé attrape le colis et l'emballe (instruction). Dès que le capteur ne détecte plus de colis ou que le quota fixé est atteint (condition fausse), le robot s'arrête. Sans capteur fonctionnel ou sans mise à jour du compteur de colis, le robot continuerait à tourner à vide indéfiniment jusqu'à provoquer une surchauffe et détruire la machine.*

2. La boucle TANT QUE (while) : Pré-conditionnement

La structure 'while' est le choix algorithmique naturel lorsque le nombre exact d'itérations n me peut pas être déterminé à l'avance et dépend d'un événement dynamique externe (une saisie utilisateur, la lecture d'un fichier jusqu me à sa fin, ou l'attente d'un signal réseau). En langage C, sa syntaxe stricte s'écrit : while (condition) { /* instructions */ } Puisque le contrôle de la condition s'effectue en amont du bloc, il est possible que les instructions internes ne s'exécutent jamais (0 itération) si la condition est fausse initialement. L'étudiant doit veiller à deux pièges fréquents : d'une part, l'omission de l'initialisation des variables de condition avant le while (conduisant à tester des valeurs indéterminées en mémoire) ; d'autre part, l'oubli de la modification de ces variables à l'intérieur de

la boucle (provoquant une boucle infinie).

• Pré-conditionnement : L'expression booléenne est évaluée AVANT d'exécuter le bloc d'instructions.

• Plage d'itérations : De 0 à N passages. Si la condition est fausse dès la première entrée, le corps de la boucle n'est jamais exécuté.

• Initialisation préalable obligatoire : Toute variable intervenant dans la condition d'évaluation du while doit posséder une valeur connue avant d'atteindre l'instruction.

Cas concret enseignant : *Exemple du contrôle de code PIN à un distributeur automatique de billets (DAB) : le système demande le code secret TANT QUE le code saisi est incorrect ET QUE le compteur d'essais est strictement inférieur à 3. Si l'usager saisit le bon code du premier coup, la condition devient fausse immédiatement et le corps de la boucle n'est pas réexécuté.*

3. La boucle FAIRE... TANT QUE (do-while) : Post-conditionnement

À la différence de la boucle 'while', la structure 'do-while' garantit que le bloc d'instructions sera exécuté au moins une première fois avant que la condition logique ne soit testée. En langage C, sa syntaxe s'écrit : do { /* instructions */ } while (condition); Cette boucle est spécifiquement adaptée à la saisie sécurisée de données (validation de bornes numériques) et à la gestion de menus interactifs. En effet, il est impossible d'évaluer la validité d'une donnée saisie par un utilisateur avant de lui avoir au moins demandé de la saisir une première fois. Le piège de syntaxe majeur réside dans l'oubli du point-virgule final juste après `while (condition);`, erreur classique qui bloque immédiatement la compilation.

• Post-conditionnement : L'expression booléenne est évaluée APRÈS chaque exécution du bloc d'instructions.

• Garantie d'exécution minimale : La boucle s'exécute obligatoirement au moins 1 fois (intervalle de 1 à N itérations).

• Point-virgule final impératif : La syntaxe C exige un ';' à la fin du while sous peine d'erreur de compilation.

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'On ne peut pas vérifier si la note entrée par un utilisateur est comprise entre 0 et 20 avant qu'il n'ait tapé la valeur au clavier. Il faut exécuter la demande de saisie au moins une fois, ce qui fait du do-while la structure idéale pour le contrôle de saisie.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant sur une ligne d'en-tête unique l'ensemble de la logique d'itération. En langage C, la syntaxe s'écrit : for (initialisation ; condition ; incrémentation) { /* instructions */ } Le fonctionnement interne respecte un ordre strict : 1) Exécution unique de l'initialisation ; 2) Évaluation de la condition booléenne ; 3) Si la condition est Vraie, exécution du corps de la boucle ; 4) Exécution de l'incrémentation/décrémentation ; 5) Retour à l'étape 2. Bien que le C autorise l'omission de certaines parties dans l'en-tête (ex: `for(;;)` pour créer une boucle infinie), la norme d'écriture à l'IUT impose de renseigner explicitement les trois expressions pour garantir une lisibilité optimale.

• En-tête condensé : Centralise l'initialisation, la condition de maintien et l'incrémentation en une seule ligne.

• Nombre d'itérations déterminé : Structure recommandée lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.

• Portée (scope) : Possibilité de limiter la durée de vie de la variable d'itération en la déclarant dans l'en-tête (standard C99).

Cas concret enseignant : *Métaphore des 5 tours de piste de l'athlète sur un stade : 'Avant d'attaquer la course, vous savez que vous devez accomplir exactement 5 tours. Vous préparez votre compteur au tour 1 (initialisation), vous contrôlez que vous n'avez pas dépassé le 5e tour (condition), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (incrémentation).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter la variable 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est jamais modifiée dans le corps de la boucle, la condition logique (par exemple `i < 10` avec `i` valant 0) reste Vraie à l'infini. Le processeur réexécute l'intérieur de la boucle sans s'arrêter, ce qui bloque le programme et consomme 100% des ressources processeur. C'est l'erreur classique de la boucle infinie.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, il existe une équivalence algorithmique totale. Tout `for (init; cond; inc) { corps; }` peut se traduire par `init; while(cond) { corps; inc; }`. La boucle `for` est simplement une convention de syntaxe (un sucre syntaxique) conçue pour rendre le code plus lisible et éviter d'oublier la mise à jour de la variable d'itération lorsque le nombre de répétitions est connu par avance.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers strictement positifs (1 + 2 + ... + N). Le programme doit impérativement sécuriser la saisie de N au moyen d'une boucle 'do-while' pour refuser toute valeur inférieure ou égale à 0, puis calculer la somme cumulative à l'aide d'une boucle 'for'. Le programme doit être rédigé selon la norme stricte de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, flèches de flot de données, style Allman).

Démarche de résolution :
Étape 1 : Déclaration des variables dans la tripartition '// VARIABLES' (`n` pour la borne supérieure, `somme` pour l'accumulateur, `i` pour le compteur de boucle). Étape 2 : Implémentation de la saisie sécurisée avec la structure `do { ... } while (n <= 0);`. La boucle s'exécute une première fois et réclame une nouvelle saisie tant que l'utilisateur fournit une valeur négative ou nulle. Étape 3 : Initialisation explicite de `somme` à 0 avant d'entamer les cumuls. Étape 4 : Utilisation de la boucle `for (i = 1; i <= n; i++)` pour incrémenter et additionner chaque valeur de 1 jusqu'à N au cumul `somme`. Étape 5 : Affichage formaté du résultat avec `printf` et retour de la fonction main avec la valeur 0.

```c
/*
  Nom du fichier : somme_n.c
  Objectif : Calcul de la somme des N premiers entiers (>0)
  Auteur : Matéi (BUT 1 Informatique - IUT de Bayonne et du Pays Basque)
  Date : 11/11/2026
*/

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // CONSTANTES
    // (Aucune constante requise pour ce traitement)

    // VARIABLES
    int n;       // Entier positif saisi par l'utilisateur
    int somme;   // Accumulateur contenant la somme cumulative
    int i;       // Compteur/variable d'itération de la boucle for

    // TRAITEMENTS
    // Saisie sécurisée de N (> 0) avec post-conditionnement do-while
    do
    {
        printf("Entrez un entier N strictement positif (> 0) : ");
        // (clavier) >> saisir >> n
        scanf("%d", &n);
    }
    while (n <= 0);

    // Initialisation de l'accumulateur
    somme = 0;

    // Calcul de la somme de 1 à N avec la boucle for
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    // Affichage du résultat final
    // somme >> (écran)
    printf("La somme des %d premiers entiers est : %d\n", n, somme);

    return 0;
}
```

Résultat : Pour un essai avec N = 5 : la boucle 'do-while' valide N (car 5 > 0). La boucle 'for' réalise 5 itérations successives : - i = 1 : somme = 0 + 1 = 1 - i = 2 : somme = 1 + 2 = 3 - i = 3 : somme = 3 + 3 = 6 - i = 4 : somme = 6 + 4 = 10 - i = 5 : somme = 10 + 5 = 15 Le programme affiche : 'La somme des 5 premiers entiers est : 15'. Si l'utilisateur avait saisi -3 au départ, le do-while aurait rejeté la valeur et redemandé la saisie.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' est à pré-condition : elle évalue la condition booléenne AVANT le corps de la boucle (0 à N itérations). La boucle 'do-while' est à post-condition : elle évalue la condition APRÈS l'exécution du corps (1 à N itérations garanties).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès la première évaluation ?

R : Le corps de la boucle 'for' n'est exécuté aucune fois (0 itération). Le programme saute le bloc d'instructions interne et passe directement à l'instruction qui suit la fermeture de la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule ';' placé obligatoirement immédiatement après la parenthèse fermante de la condition du while : `do { ... } while (condition);`.